

Projet de Compilation 2025-2026

L3 Informatique

Léo Pierrat

Mai 2026

Manuel d'utilisation

Le compilateur se construit avec :

`make`

L'exécutable produit est `bin/tpcc`. Il s'utilise de la façon suivante :

`./bin/tpcc [options] < monfichier.tpc`

Options disponibles :

Option	Description
<code>-t, --tree</code>	Affiche l'arbre abstrait sur la sortie standard
<code>-s, --syntabs</code>	Affiche toutes les tables des symboles
<code>-h, --help</code>	Affiche l'aide et termine

Codes de retour :

Code	Signification
0	Programme valide (même avec des avertissements)
1	Erreur lexicale ou syntaxique
2	Erreur sémantique

Description du compilateur

Analyse lexicale et syntaxique

L'analyse lexicale est réalisée avec Flex. Le lexeur gère les commentaires multi-lignes `/* ... */` via un état exclusif `COMM`, ce qui permet de continuer à compter les numéros de ligne à l'intérieur des commentaires. Les commentaires mono-ligne `// ...` sont ignorés par une règle simple. Les types `int` et `char` sont renvoyés comme token `TYPE` avec leur valeur, ce qui permet au parseur de les traiter uniformément.

L'analyse syntaxique est réalisée avec Bison. La grammaire implémente le langage TPC complet, y compris les types structure `struct X { ... }`. À chaque règle de grammaire correspond une action

qui construit un nœud de l'arbre abstrait via le module `tree`. Chaque nœud contient un label (type du nœud) et selon le cas un entier, un caractère ou une chaîne (identifiant ou opérateur).

Analyse sémantique

L'analyse sémantique est réalisée dans le module `sem.c`, par un parcours en profondeur de l'arbre abstrait.

Quatre tables de hachage sont utilisées (module `symb.c`, avec chaînage et optimisation move-to-front) :

- `global_table` : variables globales
- `local_table` : variables locales à la fonction en cours (réinitialisée à chaque nouvelle fonction)
- `function_table` : noms des fonctions déclarées
- `struct_table` : noms des types struct déclarés

Pré-passe. Avant le parcours principal, une pré-passe sur le nœud racine collecte tous les noms de fonctions et de structs globales. Cela permet de gérer les appels à des fonctions déclarées plus loin dans le fichier source sans générer de fausse erreur. Les quatre fonctions built-in `putchar`, `getchar`, `putint`, `getint` sont également insérées dans `function_table` au démarrage.

Erreurs détectées :

- Variable utilisée sans être déclarée
- Fonction appelée sans être déclarée
- Type `struct X` utilisé alors que `X` n'a pas été déclaré (en variable locale, globale ou en paramètre)

Avertissements :

- Affectation d'une expression de type `int` à une variable de type `char`

Génération de code

Le code cible est de l'assembleur NASM x86-64 avec les conventions d'appel AMD64. Le fichier de sortie est `_anonymous.asm` par défaut (lecture sur l'entrée standard).

Structure du programme généré. Le fichier assembleur contient :

- Un point d'entrée `_start` qui appelle `main` puis effectue le syscall `exit` avec le code de retour
- Les quatre fonctions built-in d'E/S écrites directement en assembleur
- Les fonctions du programme source
- Une section `.bss` pour les variables globales

Expressions. Les expressions sont évaluées par une approche à pile : chaque sous-expression empile son résultat, les opérateurs dépilent leurs opérandes dans `rax` et `r10` et empilent le résultat. La division utilise `idiv` après `xor rdx, rdx`. La multiplication utilise la forme à deux opérandes `imul rax, r10`.

Booléens. Les expressions booléennes (`&&`, `||`, `!`, `==`, `!=`, `<`, `>`, `<=`, `>=`) utilisent l'évaluation paresseuse (court-circuit) avec des labels. Chaque nœud booléen reçoit un `label_true` et un `label_false` et y saute directement sans passer par la pile.

Fonctions. Chaque fonction génère un prologue (`push rbp ; mov rbp, rsp ; sub rsp, N`) et un épilogue (`mov rsp, rbp ; pop rbp ; ret`). Les arguments sont passés dans les registres `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` et copiés sur la pile locale dans le prologue. Les variables locales sont allouées avec des déplacements négatifs par rapport à `rbp`.

Fonctions I/O built-in. Les quatre fonctions sont écrites directement en NASM en utilisant uniquement les syscalls Linux (`rax=0` pour `read`, `rax=1` pour `write`, `rax=60` pour `exit`). Seuls les concepts vus en cours sont utilisés.

Limitation. L'accès aux champs de structures (`a.x`) n'est pas implémenté en génération de code. Les types `struct` sont déclarés et vérifiés sémantiquement, mais le layout mémoire et la lecture/écriture de champs ne sont pas gérés.

Difficultés rencontrées

Convention AMD64 — registres callee-save. Le registre `rbx` est callee-save et ne peut pas être utilisé comme registre scratch sans être sauvegardé au préalable. Tous les registres temporaires utilisent `r10` (caller-save), qui peut être écrasé librement.

Instruction `idiv`. L'instruction requiert que `rdx` soit mis à zéro avant la division (`xor rdx, rdx`), sous peine de comportement indéfini. Cette contrainte s'applique aussi bien pour la division que pour le modulo (le reste est dans `rdx`).

Évaluation paresseuse. La gestion des labels pour `&&` et `||` est délicate : chaque nœud doit propager ses labels `label_true` et `label_false` récursivement sans les mélanger avec ceux des nœuds voisins. La solution est de passer ces labels en paramètre à `write_asm_bool`.

Pré-passe pour les fonctions. Sans pré-passe, appeler une fonction déclarée après son point d'utilisation dans le fichier source provoquait une fausse erreur sémantique. La pré-passe résout ce problème en collectant tous les noms avant le parcours principal.

Prologue et allocation de la pile. La taille de la zone locale n'est connue qu'après avoir parcouru toutes les déclarations locales. L'instruction `sub rsp, N` est donc générée après le parcours du corps de la fonction, en utilisant la valeur finale de `current_offset`.